把决策耗时，
作为算法成本直接优化

本轮以此前的新基础控制器为起点，比较13个实现/决策预算候选。最终保留动作容量预留、有效历史范围遍历和阶段稳定后的重复检查消除；未因代码看起来更复杂或理论运算更少就接受改动。

冻结后336个新输入 / 各3次	原实现	CPU优化版
累计决策耗时 / 微秒	224,439.16	220,882.84
全部miss次数	98,915	98,915
相对朴素基线的Score几何比	0.651321	0.652027
单组公式得分均值 / 10	2.0016	2.0014

本轮能成立的结论

累计决策耗时变化为降低1.58%。610个开发输入和336个新输入的严格回放显示，最终版本的动作顺序、读写字节与hit/miss均与原实现一致。因此策略的I/O收益没有被拿来掩盖CPU退化。

计时变化有噪声，不能据极短样例的微小比值宣布普遍提速；大输入另作五次配对重复。若某个场景退化，后面的逐场景表完整列出。所有Score包含公开引擎原样测得的决策时间。

完整Score的几何比和单组公式均值没有同步改善，规模重复也包含退化。因此本轮暂不替换默认策略，CPU文件作为可选微优化候选交付。

交付

design/MySpillScheduler_cpu.cpp：单文件、标准库、C++14、无main、默认无参数实例化。未覆盖此前两个提交文件。详细候选、输入种子、审计与逐次运行均位于design/decision_cpu_20261026。

这里的原实现指上一轮基础控制器flags=351，不是旧学习冲分版，也不是朴素基线。所有结果是本机合成负载，非官方隐藏数据成绩。

优化的判断标准：花费的CPU必须有回报

直接比较完整代价

Score = I/O时间 + p×miss次数 + 决策时间。若新决策节约的I/O为GI、节约miss数为GM、额外CPU为DC，则净收益为GI+p×GM-DC。少一次miss不一定值得额外搜索，少一点CPU也不一定值得增加读盘。

真实p没有出现在SystemState中，生产策略不能读取它。本地评测知道输入参数，可以核算真实Score；在线的保证只能使用题面允许的范围，例如p>=1，或承认所用尺度是启发式。

路径	成本取舍
必需动作	miss末尾LOAD目标、overflow释放足够空间不能为了省CPU而跳过
无额外落盘的下一目标预取	已知下一目标且空间足够，保留低成本检查；不进行多轮搜索
历史维护	减少频率可能省CPU，却会改变热度和下一次淘汰；必须测完整Score
等价代码优化	在相同动作/I/O/miss下，Score差异直接来自决策时间；优先选择这一类
提前搬动相同计算	计算仍在其他回调里计时；只有省去重复工作或一次回调才可能获益

本轮没有做的“提速”

没有修改周期B，没有移动getSystemState出计时区间，没有修改HashJoinEngine或main，没有增加命中回调，没有关闭决策计时，没有读取隐藏罚时或后续完整操作。

自适应继续保留

首次实际miss初始化，后续仍根据阶段变化、Probe/Miss窗口和短间隔读回反馈重新判断。最终优化版没有固定降低观测频率，也没有关闭跨算子调度或下一步预取来换取表面CPU下降。

先测出时间花在哪里，再决定优化边界

低频采样诊断

诊断版对约1/64回调随机采样内部耗时，区分overflow、miss和periodic。采样本身增加开销，且调用形态会影响编译，因此下表用于定位；最终Score均来自不带诊断包装的公开引擎。

场景	回调	次数	采样内部均值
spilled_hot_B1	periodic	1,000,575	28.4ns
resident_pressure_B1	miss	22,088	2369.5ns
resident_pressure_B1	periodic	50,512	39.8ns
resident_pressure_B1000	overflow	257	1642.4ns
resident_pressure_B1000	miss	21,869	2405.0ns
maxN_maxQ_B1_hits	periodic	4,999,992	18.8ns

框架固定成本是关键限制

公开参考代码在计时lambda内部构造SystemState快照，包含operators和partitions的复制/析构。全驻留B=1输入上，内部周期函数只有约20ns，总决策计时折合每次回调却约0.28微秒。这里的差额也含计时与诊断影响，不能全部精确归因于一次复制。

这意味着把一个20ns函数继续缩短，无法消除约280ns的完整回调成本。题面要求保留框架，所以优化优先级应当是减少重复计划与内存分配，并在合法预取确实避免miss时减少额外回调。

只看内部函数的百分比，会高估整体Score提升；只看墙上时间，则混入输入解析和操作模拟。本报告分别记录这三个口径。

最终保留的三个改动

一、动作结果只分配所需容量

原单SPILL计划返回容量为1的vector，随后追加下一目标LOAD、当前目标LOAD，可能经历1→2→4的重复扩容。现在Probe计划预留两个LOAD的位置；空计划也预留至多两个LOAD。动作顺序和值不变。

如果只有一个SPILL加一个LOAD，原路径通常两次分配，新路径一次；若还预取下一目标，原路径可能三次，新路径一次。API仍要求返回std::vector，因此没有宣称消除了全部分配。

二、历史到期检查只遍历实际Q=K×P条目

原实现每次事件评估都检查512项ghost。实际可能只有2、8或16个分区。现在按当前固定的Q遍历；未使用的条目从未被赋值，跳过它们不改变反馈与规则切换。

三、阶段已经稳定时省去重复扫描

算子只会从BUILD转到PROBE。当所有算子都已进入PROBE，阶段掩码永久不变。之后继续检查窗口和读回信号，但不用在每次miss里重新遍历算子确认阶段。混合BUILD/PROBE期间仍完整检查。

为什么没有默认采用大型驻留索引

增量驻留表虽然减少扫描条目，却增加ID解码、间接访问、维护成本和代码体积；本机实测没有净收益。栈上计划、编译期固定配置和整分区代价缓存也出现类似情况。操作数更少，不等于接口总耗时更少。

这些优化保留原先的完整比较和浮点计算；最终文件没有使用倒数替代除法，也没有依赖编译器扩展或fast-math。

被拒绝的方案，也属于这轮的实验证据

候选	实测与取舍
编译期配置 / 栈上计划	动作一致，但CPU敏感负载上累计决策时间更高；不采用
固定每16次更新历史	610个开发输入中256例动作改变；Score几何比从约0.5411变为0.5454，更差；不采用
动态CPU维护预算	根据可见I/O成本下界和采样CPU决定1~64的更新步长；21例动作改变，没有稳定净收益；不采用
增量驻留索引	全PROBE后维护驻留集合和固定大小；维护/解码成本抵消减少扫描的收益；不采用
不可变读写费缓存	数学化简只在全部候选可覆盖缺口时成立；整体未显示稳定收益；不采用
倒数外提 / 过滤比值比较	开发回放没有发现动作差异，仍涉及数值与额外分支；没有稳定净收益，不放入最终文件
高频短路径与冷函数分离	标准C++间接调用隔离大函数；实测没有稳定优于小范围优化，不采用

动态采样为何没有被“创新性”强行保留

它确实实现了在线感知：按实际I/O/Miss变化和测得维护时间调整步长。但这种感知同时改变热度样本，会影响后续淘汰。满足自适应形式不等于Score更优，必须用消融决定是否启用。

开发集与独立验证分开

以上方案在历史610例及CPU定向输入上比较。最终源文件先冻结，再用新系统熵生成336例；新例没有用于继续调参。CPU/500万操作输入是重复测量与规模回归，不能称为独立随机留出集。

五轮CPU与规模对照：明确区分耗时和Score

以下每例重复5次，交替运行原实现与最终版本，取各自中位数。两者读写字节和miss完全相同；表中百分比为相对原实现的降低，负数表示退化。

场景	决策时间降低	完整Score降低	新墙上时间
spilled_hot_B1	-0.59%	-0.57505%	0.62s
resident_pressure_B1	-0.71%	-0.04542%	0.11s
resident_pressure_B1000	0.11%	0.00573%	0.09s
maxN_maxQ_B1_hits	0.70%	0.69906%	3.09s
maxN_mixed_maxQ	-0.79%	-0.00047%	4.90s
maxN_growth_4GiB	1.82%	0.00317%	2.13s

解释结果时的限制

CPU主导输入中，少量纳秒级开销会累计成可见Score差异；I/O主导输入中，即使决策时间减少几个百分点，完整Score改善也可能不足0.01%。大分区读盘成本不会因为小代码优化而消失。

即使动作相同，也不能保证每个输入实测时间都更短。短样例可能只花不足一微秒，输出四舍五入和系统调度会放大比值；不能把一次11倍的极短计时比值当成稳定算法退化。

采用标准

最终版本优先保持动作质量、减少确定存在的重复分配与扫描。若某个评测机上CPU结果不同，应按同样的配对测试重新判断；本轮没有官方机器或隐藏输入的结果。

冻结、核验与复现入口

检查	结果
新输入	336例，20类访问分布×8、128端点组合、6类针对性场景×8
行为等价	610开发例+336新例：动作顺序、写字节、读字节、Hit/Miss一致
动作合法	无OOM、无BUILD LOAD、overflow不落盘当前目标、miss末尾LOAD当前目标
公开框架	只替换调度器include与实例化；计时区间完全保留
提交接口	g++ -std=c++14 -O2；单文件；标准库；默认无参数实例化
规模限制	五轮CPU/规模测试中每次运行均在60秒以内
冻结摘要	d4242468bcec0ff4e00fb7c0e6fa36dbe75f32eb5d7455e15ee387e765f7da8c

文件

MySpillScheduler_cpu.cpp：可单独替换提交的CPU优化候选；不要与其他同名类文件同时编译。
decision_cpu_20261026/README.txt：结果、候选、边界与复现说明。
validation_runs.jsonl：新336例三次完整Score。
scale_cpu_runs.jsonl：六场景五次配对原始计时。
*_strict.jsonl：独立动作合法性与等价回放。

当前算法策略

BUILD维持跨算子内存回收；PROBE仍用观测代价、窗口/事件反馈和热度规则；next_op保留低开销预取及相同BUILD计划的提前执行。没有默认开启上一轮风险较大的目标提前落盘，也没有在线训练大模型。

下一步值得优化的对象

只有新的剖析或工作负载显示收益时，再扩大搜索。尤其应观察“减少一次miss得到多少I/O/罚时收益”和“额外计划实际用了多少CPU”。本轮失败候选已保留，避免下次重复投入。

原始输入、源文件摘要、编译信息与检查结果保存在final_audit.json；源文件未在独立验证后继续调整。